

YAMLを1行変えただけでCIのコストが大幅に下がる!?

～Blacksmithをプロダクトで試してみた～

2026/09/30 PHP勉強会東京

おさない

**たった数行の修正でCIが速く・安くなる。
そんな銀の弾丸、あると思いますか？**

今日持ち帰ってほしいこと

- CIのコストダウン手段としてGHA runnerを Blacksmith にするアプローチを紹介する
- プロダクトに入れた結果、効いたケースと効かなかったケースがあるため、それぞれの事例を紹介する

アジェンダ

1. そもそもBlacksmithとは？
2. プロダクトに導入した結果

Blacksmithとは

- Blacksmithが提供するGitHub Actionsの代替runner
 - <https://www.blacksmith.sh/>
- CIのコスト削減、速度向上に寄与する

GitHub Actionsのrunnerとは

- GitHub Actionsワークフローでジョブを実行する環境
 - GitHub公式が管理・提供するGitHub-hosted runner
 - 自分でマシン等を用意するself-hosted runner

CIのコストを下げるには

CIのジョブにかかる料金 $\text{Runnerの単価} \times \text{実行時間}$

CIのコストを下げるアプローチ：

1. Runnerの単価を下げる
2. 実行時間を縮める

Blacksmithは双方に働きかけることができる

1. 分単価が安くなる

	分単価
GitHub-hosted Linux 2-core	\$0.006/分
Blacksmith Ubuntu x64	\$0.004/分

- Blacksmith公式: GitHubの分単価より **33%安い**
 - <https://www.blacksmith.sh/pricing>

2. 性能が上がる

Blacksmith公式によると、

- GitHubのRunnerより 2倍高速なハードウェア
- キャッシュのダウンロードが 4倍高速
- 実行時間が半分なら、単価33%安と合わせて最大 67% 削減

導入のきっかけ

- push数・PR数増加によるCI料金の高騰
- 下記記事に釣られて導入
- 3つのプロダクトのテストや静的解析で試してみることに

**YAMLを数行変えただけでGithub
Actionsの料金を70%削減できて実行時
間まで減った話**



COUNTERWORKS テックブログ
shim

 **Zenn**

導入手順

1. `app.blacksmith.sh` で GitHub Organization と連携
2. `runs-on` を Blacksmith の Runner に変える

```
runs-on: ubuntu-latest
```



```
runs-on: blacksmith-2vcpu-ubuntu-2404
```

※制約：GitHub Organizations が必要

プロダクト A / Bでは効いた

成功 run の中央値。どちらも 2vCPU

	実行時間	1回のコスト
A の テスト	9.7分 → 3.9分	\$0.192 → \$0.056
B の テスト	7.5分 → 4.0分	\$0.048 → \$0.020
B の静的解析	3.2分 → 1.8分	\$0.060 → \$0.024

実行時間が平均50%、コストが平均63%削減 → 採用

プロダクトCでは効かなかった

	導入前	Blacksmith 2vCPU
テスト本体	38秒	15秒
setup-php	10秒	78秒
ジョブ全体	69秒	103秒
3ジョブのコスト	\$0.024	\$0.024

- プロダクトCで利用していたsetup-phpの所要時間が大きく増加。
- テスト本体の速度は向上
- job全体が逆に遅くなったため採用見送り

なぜ setup-php だけ遅くなる？

- shivammathur/setup-php のself-hosted runnerにおける制約
- <https://github.com/shivammathur/setup-php/issues/1056>
- GitHub-hosted — 入っているライブラリが既知なので、ビルド済みキャッシュを展開するだけ
- self-hosted runner — 不足ライブラリを apt で都度入れるので時間がかかる

最終的なBlacksmithの採用状況

- プロダクトA / Bは採用
 - CIの所要時間が縮み、コストも減った
- プロダクトCは見送り
 - テスト本体は速いが setup-php が膨らみ、全体の時間は悪化、コストは横ばい
- 切り戻す際はGHAのruns-onを戻せばよい

まとめ

1. CIコストは 単価 × 実行時間。Blacksmithは双方にアプローチできる
2. 導入は Organization 連携と、`runs-on` の変更
3. 効果はCIで実測して判断。切り戻しも `runs-on` 1行の変更

「次は、Blacksmithの導入から、始まる」

PHPカンファレンス関西2026開催

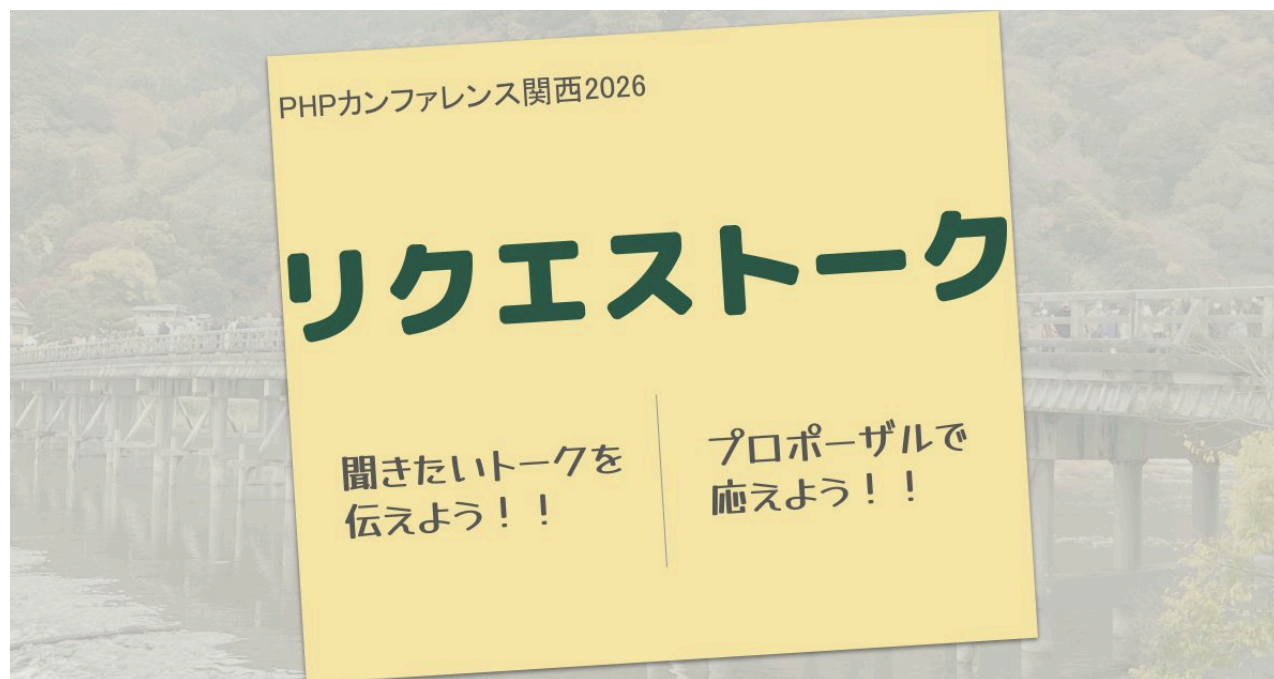
- スポンサー募集中！
- プロポーザル、リクエストトーク絶賛募集中！



<https://2026.kphpug.jp/>

新企画「リクエストーク」

- PHPカンファレンス関西2026で「聞きたいトーク」を聞かせてください
- 「聞きたい！」をプロポーザル出したい方にお届けします！
- 「新たなコミュニティへの携わり方」を作ります！



今年もプロポーザル作成ワークショップを行います！

- 参加者募集開始！
- オンライン見学枠もあります！
- <https://phpkansai.connpass.com/event/407721/>



PHPカンファレンス愛媛2026

- 2026/10/03(土) 愛媛県松山市で開催！
- チケット発売中！



<https://phpcon.ehime.jp/>

PHPカンファレンス新潟2026

- 2026/11/14(土) 新潟県新潟市で開催！
- チケット販売中！当日スタッフ募集中！



<https://phpcon.niigata.jp/>

自己紹介



おさない

Twitter: @000sak000

References

1. GitHub Actions Billing — Linux 2-core は \$0.006/分
<https://docs.github.com/en/billing/concepts/product-billing/github-actions>
2. Blacksmith Pricing — Ubuntu x64 \$0.004/分、33%安、2倍速、最大67%
<https://www.blacksmith.sh/pricing>
3. Blacksmith Quickstart — 連携と `runs-on` の対応
<https://docs.blacksmith.sh/introduction/quickstart>
4. setup-php Issue #1056 — Third-party Runner で setup-php が遅い
<https://github.com/shivammathur/setup-php/issues/1056>